

Alarm clock

Day 1

7/08/2026

I wanna get into designing and making things, both to spend my time productively rather than lounging around, and to apply the knowledge I've learned in my high school courses. I consulted ChatGPT for initial ideas, but the structure and implementation will be my own.

Features

- Display time
- Display date
- Set alarm for date/time, make a vibration and noise
- Turn on / off
- Calibrate time
- Buttons?
- Knob?
- Maybe a small battery (like computers have) to keep track of time when powered off

What I think I'd need, physically:

- Shell (3D Printed?)
- Board
- Wiring
- Display
- Switch
- Knob
- Motor
- Speaker
- Coin Battery

I'm thinking a simple screen with the time, date, and a simple + button to add a timer. I feel like the simplistic look will make it clean and simple to look at.

The date above will act as a header, likely adjusted to the left. The time in the center, and then fill the bottom right space with the button.

Since I need a way to calibrate time (probably), as I don't know how I'd make it automatic, I'll pull up a time selection screen when you click the date. I feel like if I made it the time and the date, it'll show up too often on accident, since that'll be most of the screen.

For the time selection screen, I simply made the time and date the same height, and put them side by side with arrows. I was originally thinking a scrolling-type wheel selection screen with both the time and date, like a physical lock, but after sketching it, it looked too cluttered. That's why I replaced them with simple arrows. After sketching that out, I realized it would function

double as an alarm screen, reducing the amount of code that would be needed; I decided to reuse the sketch for both.

To test the limits of what I can do, I want to add physical interactions. I think a knob with a button built into it is enough. Clicking down could switch between elements, like from month, to day, to year, to hour, etc. Twisting the knob could act as a substitute for clicking the onscreen arrows, moving each element up or down.

However, for my first version, which is digital only, that won't matter. The arrows and onscreen buttons will be just fine.

Here's my current, first version sketch for both.

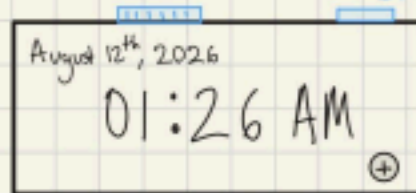
Alarm Clock

- Time
 - Date (full name. ex. August 12th, 2026)
 - Hour
 - Minute
- Alarm
 - Date, Hour, Minute
 - Vibration
 - Sound

- Time Calibration
- On/off button

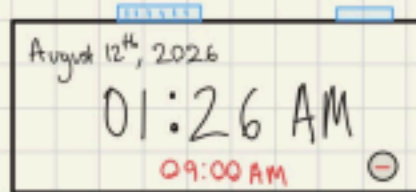
• Physical

V1

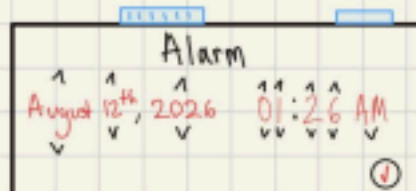
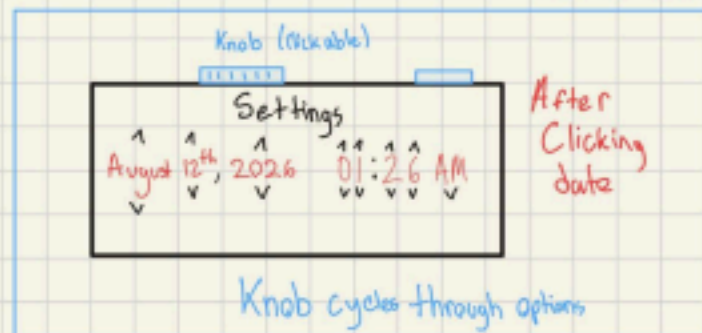


Power Button

input
latch
clicking



Alarm
Set for 9AM



Knob cycles through options

ChatGPT feedback on questions I should ask myself:

“

- Can alarms repeat?
- What happens if an alarm goes off while you're editing settings?
- What happens if someone turns it off while an alarm is ringing?
- What happens if the backup battery dies?

“

Right off the bat, I can see key areas I missed off my initial planning. Whether the settings are discarded or saved if an alarm goes off while in the menu is important. My first thought was just to drop the settings, but that's annoying.

My second thought was just to try to save them but not apply them, then reopen the menu once the alarm is dismissed. I think I'll take that route.

I also didn't plan an alarm menu itself, which I need to do.

I'll try to think of answers to these questions in depth tomorrow, and make a reflection doc.

ChatGPT also told me about a diagram called a state diagram, which is how a user moves through menus/states through their interactions. That seems fairly useful, and I'll look up their basic properties tomorrow and jot down some notes. I need to think about how the "states" connect to one another— this is even more apparent after considering the settings -> alarm going off connection.

Side note: When I eventually get to a physical version, I want it to be inexpensive to make (ideally less than \$30) so I can give a few to my friends.

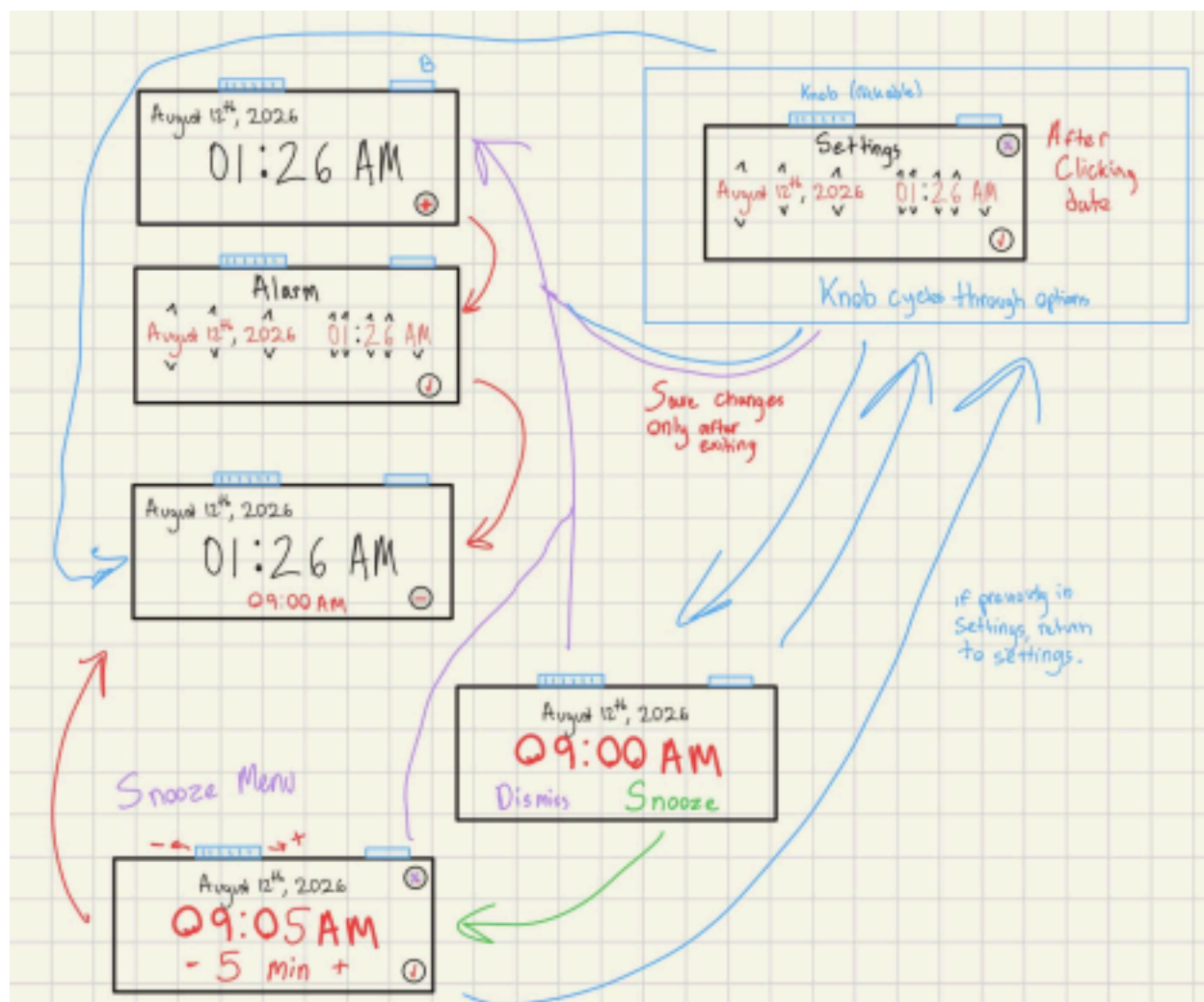
Day 2:

7/09/2026

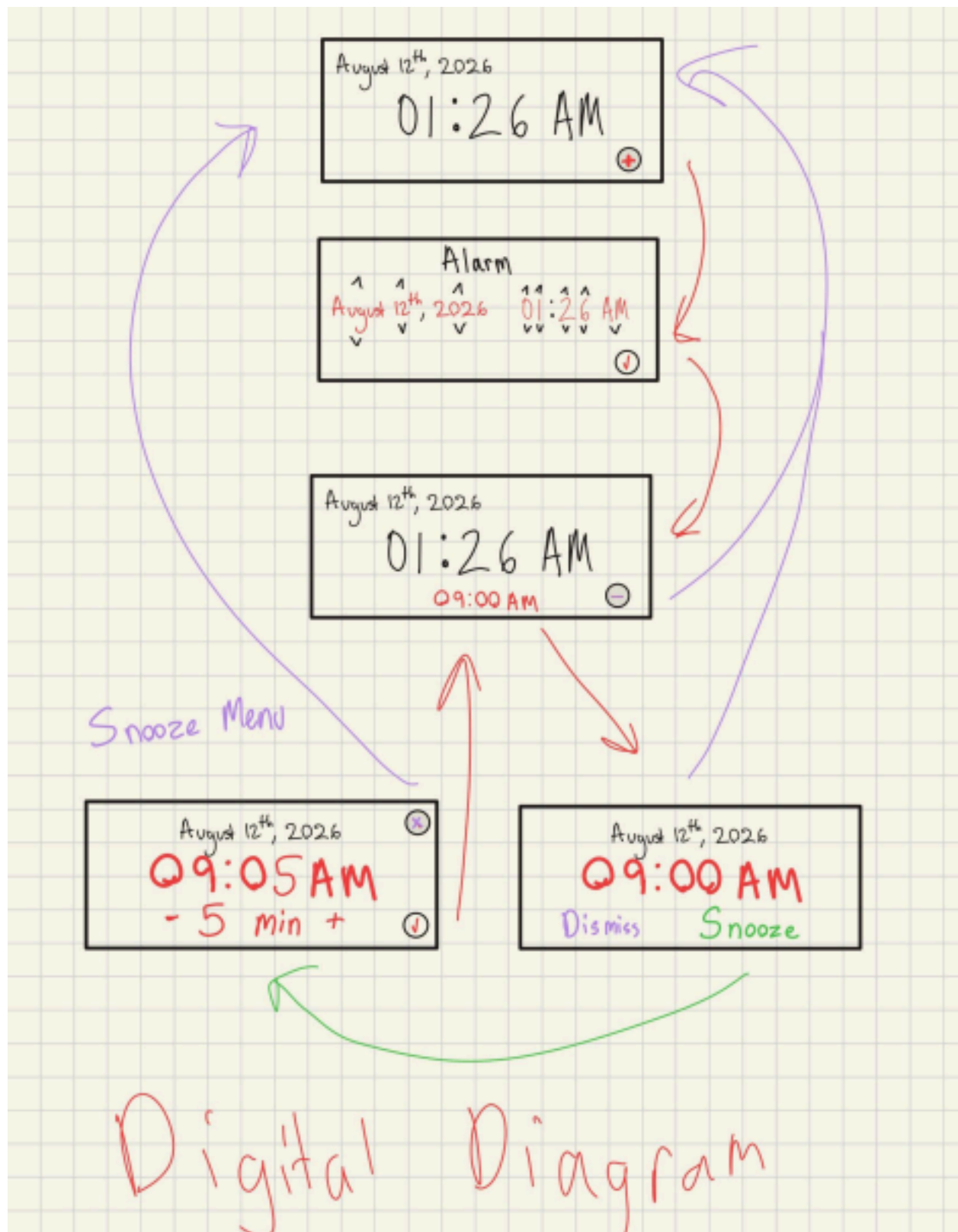
I'm currently drawing the state diagram. I've realized the clock needs a snooze feature, so I've designed that menu, and worked out that if the alarm goes off while in the settings, you should be taken to the normal alarm menu, however after dismissing it or snoozing, you should be returned to the settings menu exactly as you left it. This would require a way to track the last page you left off on. This may also be helpful in general if more intricate page systems are added in the future.

Along with the settings menu being as you left it, I feel it'd be more simple if the settings weren't even applied to the overall clock time until the exit button is pressed in the settings menu. For the state diagrams, I've prepared one for the physical version, which is much more messy/complex due to the settings page, and one for the digital version which shouldn't (maybe) need a settings page. For now at least.

Here they both are:



Physical + Digital State Diagram



For everything to work as I've described it, I'd need to have data stored on file permanently, and data stored temporarily before being purged or being used to overwrite other data. For example, permanent data would have to be the date and time configuration, and the list of set alarms. For now it's just one alarm, but I'd like to expand it after my first version. For temporary data, that'd be the configuration of the settings menu before the confirm button is pressed, and same with the alarm setting and snooze menu.

Alright, so how it's actually gonna work. I think I should have a loop repeat every second (if that's possible) which upticks the seconds of the clock's time by 1 (which will probably be a hidden variable, since there's no spot for seconds in my design, at least as it is right now) when that runs at 59 seconds, it'll set it back down to zero and uptick the minute by one, so on and so forth for hours, days, months, years, etc.

For the months, I think I'll do it numerically internally, then have a list which displays the month's name based on the index minus one (well that's how I'd do it in AP Computer Science A (CSA) anyway).

For the alarm system itself, I think I'll have an alarm object and a time object, and check every second if their date and time values are equal. When they're equal, a function could be called to change the clock's display to the alarm screen, and then play a tune, which is stopped when the alarm screen is dismissed or snoozed.

I have to research how to make all of this still, but this is how I'd try to approach it with my current knowledge.

Day 3

7/10/2026

Last night, I spent a few hours researching C# documentation after being directed there when searching how to make a small windows app. I wanted to make a windows app to model my alarm system, with the states I planned out. It took a lot of time and effort, especially with figuring out how to loop the time updating without bricking the program.

This problem arose due to me creating an endless loop in a function, so the control was never returned to my Main() method. Taking a step back, c# was very similar to my experience with Java through AP CSA, which was unexpected for me.

I learned about asynchronous modifiers, asynchronous timers, input events, and how vast the c# built-in library is. My code suffered from many problems with input handling, such as simply declaring the ConsoleKeyInfo object at the top of the class, instead of where needed. To solve these issues, I opted for a modular approach using functions to separate each piece of the programming. That led me to have PullTime(), ScreenRefresh(), InputManager(), and Main() functions. PullTime is in charge of taking the time and formatting it to look nice, ScreenRefresh is in charge of drawing the menus, InputManager takes user input and updates the current state

of the program accordingly, thereby changing how the ScreenRefresh's control statements flow, and Main stitches them together in a loop.

Here is my code for the end of last night:

[AlarmClockV1](#)

Today, I'll try to remake it with a more concise structure and readability.

Alright, I rewrote (which I've been told is called refactoring) the code, and afterwards I feel much better about the skills I've practiced from doing so. I feel like I understand the code better now, and can clearly pick apart the connections between each function. This means that I feel it'll be easier for me to add new features, which I'll do tomorrow, and to remember the way I coded the program from memory.

I also removed lines of code which I felt were unnecessary, and renamed variables and functions to be easier to recognize their purpose. An example of this was the renaming of the `PullTime()` function to `Pull_Formatted_Time()`. This distinction makes it clearer for me, and hopefully others, to understand that I should use that function specifically to format the time as I plan to on the main page.

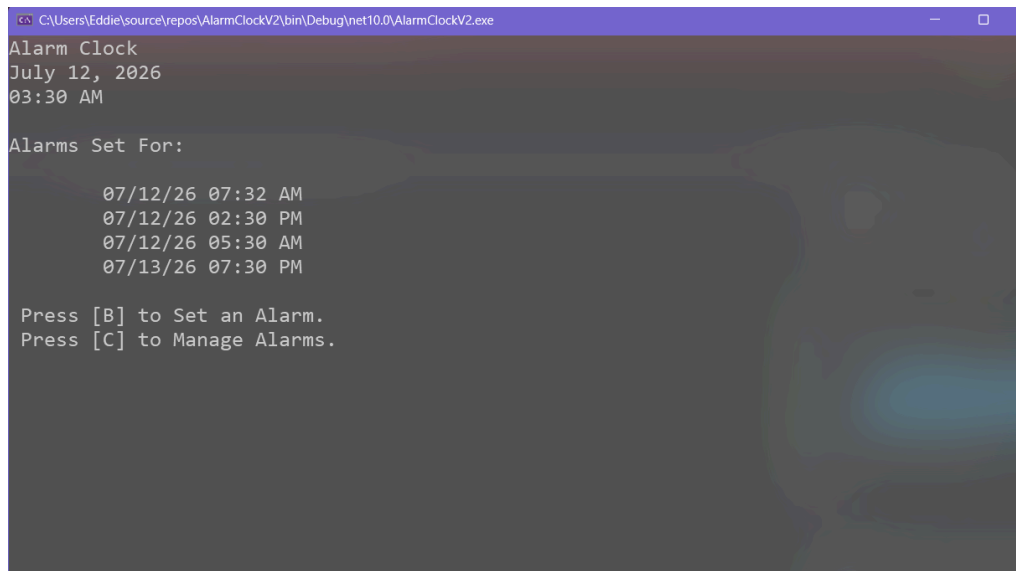
Day 4

7/11/2026

Today I worked on adding the ability to create multiple alarms to track at once. With this, I had to rewrite much of the program again. Firstly, I had to change the `DateTime` object for the singular alarm to a `List<>` object. This object felt pretty similar to the `ArrayList<>` object I learned in Java from taking AP CSA, so changing the syntax around that itself wasn't too difficult.

Firstly, I changed my simple check inside of the `Main()` method for a matching alarm to an individual function, `Alarm_Manager()`. I did this using a `foreach` loop which checked each alarm against the formatted time, as my singular `if` statement did before. I chose to check against the formatted time rather than the exact time using the `compare` method of the `DateTime` objects as I don't know how much precision is required for that to evaluate to `true`.

To continue, I required a way to obtain the alarm's index, which I decided to build into the `Alarm_Manager()` function, which I'm not sure was the right choice, but makes the name more consistent. I decided to add an `int` parameter to the `Alarm_Manager`, which decides which behavior it will take, returning a `true/false` statement for alarm time equality, printing all alarms (which I'll discuss in a moment), or returning the index of the matching alarm. Looking back, this may have been too grand for one function, and I may have to split them apart in the future. Anyways, I needed to obtain the index of the matching alarm so the program would know which alarm to delete after it went off.



```
C:\Users\Eddie\source\repos\AlarmClockV2\bin\Debug\net10.0\AlarmClockV2.exe
Alarm Clock
July 12, 2026
03:30 AM

Alarms Set For:

    07/12/26 07:32 AM
    07/12/26 02:30 PM
    07/12/26 05:30 AM
    07/13/26 07:30 PM

Press [B] to Set an Alarm.
Press [C] to Manage Alarms.
```

However, my main issues arose when I had to figure out how to remove individual alarms based on the **user** entering data.

To continue implementing this set of features, I needed to create a fourth state using the `Current_State` string, the Alarm Manager. In the Alarm Manager, named `Alarm_Manager_Menu`, I wanted to have the full list of alarms displayed, along with their indexes, and the ability to enter the index at the bottom. In order to do this, I needed a function to print all of the alarms in a neatly formatted manner and to figure out how to adapt my program's preexisting Input Manager to work with input that wasn't a single character. In the end, I had to work with single character input, as the constantly refreshing loop I started with limited my options for long inputs.

After trying and failing to use the `Console.ReadLine()` method, I implemented a system concatenating the single character input onto a string, before converting it to an int when the user presses enter. This allowed me to allow the user to input the index of the alarm to be deleted. After touching up on the UI to make the function of the screen clear (in fact, I had to change formatting while writing this, as I forgot to add AM/PM to the main and manager screens), I looked into preventing crashes which could easily occur from invalid user input.

Looking inside of my `Input_Manager` function, I sought to add constraints to when the user could finalize their deletion. I came up with 3 main conditions that their data should satisfy:

1. It couldn't be negative. After considering the fact that I had already restricted their input using `char.IsDigit()`, I realized that this didn't really have to be considered much.
2. The integer must be lower than the size of the List. I did by comparing the parsed integer (since the user's input is originally a string) with the difference of the count of the list and 1.
3. The input can't be empty. I did this by comparing the input with an empty string `""` and taking the reverse boolean using the `!` operator.

```
else if(Pressed_Key.Key == ConsoleKey.Enter && !Delete_Index.Equals("") && int.Parse(Delete_Index) < Alarms.Count )
{
    Alarms.RemoveAt(int.Parse(Delete_Index));
    Alarm_Status_Validator();
    Console.Clear();
    Delete_Index = "";
    Current_State = "Main";
}
```

This made it so the user could only enter digits, and they could only submit valid data, as far as I'm currently aware. Hopefully, this will help tame unnecessary crashes in the program.

Furthermore, in an attempt to reduce the amount of repeated code, I created the `Alarm_Status_Validator()` method, seen in the screenshot above. This is a short method which simply compares the Alarm List's value to 0, and sets the static `Alarm_Status` boolean to true if it is larger than 0, and false otherwise. This helps to control the UI of the main method, unlocking the option to access the `Alarm_Manager` menu if true, among other things. Essentially, it tracks if it is true or false that there are active alarms.

```
5 references
public static void Alarm_Status_Validator() {
    if (Alarms.Count == 0) { Alarm_Status = false; } else { Alarm_Status = true; }
}
```

Lastly, after the several hours of time I poured into this, I decided to take a break and focus my efforts elsewhere. I watched a tutorial on basic git management, and uploaded my first commit to a repository for the Alarm Clock project. In that repository, I uploaded this second version of my code (although it is labeled V1), then wrote this entry into my project journal.